

Dealing with heterogeneity through limited duplication for scheduling precedence constrained task graphs

Savina Bansal^{a,*}, Padam Kumar^b, Kuldeep Singh^b

^aDepartment of Electronics Engineering, GZS College of Engineering & Technology, Bathinda (Pb.)-151 001, India

^bDepartment of Electronics & Computer Engineering, Indian Institute of Technology, Roorkee-247 667, India

Received 9 September 2003; received in revised form 27 October 2004

Abstract

Scheduling is one of the vital design issues for heterogeneous computing systems. The work available in literature, by and large, focuses on the scheduling of ‘metatasks’ (set of independent tasks with only few data dependent subtasks) in such an environment. A majority of complex scientific/engineering applications, however, fall in the category of precedence-constrained task graphs. Scheduling of such graphs is generally limited to list-based techniques. Further, diverse performance metrics and heterogeneity models, as adopted by various researchers, make the comparison process quite inconclusive. We have made an attempt to categorize heterogeneity models and performance metrics so as to have better understanding and to facilitate a more uniform platform for developing and comparing such algorithms. Further, a generic scheduling strategy is presented, which improves the performance of list-based heuristics with the addition of limited duplication. A comparison of state-of-the-art scheduling algorithms is next performed using an A-Cube performance model that has been suggested to study the behavior of an algorithm, application and architecture in the presence of heterogeneity. Depending upon the type and extent of heterogeneity, performance of an algorithm is found to degrade with heterogeneity as the penalty imposed for any injudicious move on the part of heuristic tends to be more severe for heterogeneous computing environment in comparison to its homogeneous counterpart. However, the proposed heterogeneous limited duplication algorithm, having its roots in our earlier suggested SD algorithm, succeeds substantially in overcoming these ‘stresses’ and ‘strains’ of heterogeneity, and retains the best features of duplication without compromising much on the complexity front.

© 2004 Elsevier Inc. All rights reserved.

Keywords: Heterogeneous computing; Scheduling algorithm; Duplication heuristic; Static scheduling; Distributed computing

1. Introduction

Parallel computing machines deliver their best performance provided the parallelism embedded in a problem matches the machine architecture [8,10,21]. However, the parallelism available in scientific and engineering applications, which have been shown to possess a vast potential for concurrent computing [11,13], may or may not be homogeneous in nature. Amdahl’s law imposes another performance-limiting barrier on parallel machines; as for

most of the real application parallel programs, 10–30% of total computations are sequential in nature. The presence of diverse type of parallelism and sequential bottleneck part in application programs paved the way for a variety of heterogeneous architectures over the last decade, and heterogeneous computing (HC) is emerging as an economical substitute for high-performance computing [8,10].

Scheduling is one of the basic design issues in an HC system, and relates to minimizing the overall execution time (makespan) of a given application program. It is, in general, a well-known NP-complete problem [20]. Consequently, heuristic-based algorithms, suggested from time to time [4,20,22,23], tend to sacrifice optimality for the sake of efficiency. In static or compile-time task scheduling, the application task graph is assumed to be known a priori.

* Corresponding author.

E-mail addresses: savina_bansal@indiatimes.com (S. Bansal), padamfec@iitr.ernet.in (P. Kumar), kds56fec@iitr.ernet.in (K. Singh).

Task profiling and analytical benchmarking tools play an important role in providing the right estimation of costs on nodes and links, prior to scheduling [8,21]. The exact estimation of costs, at times, may not be possible due to its dependence on multi-variable parameters and on run-time input data. In contrast, dynamic scheduling techniques are more likely to make realistic decisions at run-time by considering actual execution costs, and may offer better load balancing and system utilization [15]. Nonetheless, static scheduling is considered in this work, as it does not incur run-time scheduling overheads as is the case with dynamic scheduling techniques and hence, it can afford to use sophisticated scheduling heuristics to generate much better schedules at compile-time. These techniques offer a good ex-post facto tool for evaluating dynamic heuristics and may be used as part of scheduling adviser tool in heterogeneous network management systems.

A wide variety of static task scheduling heuristics (e.g. list-based, duplication-based, and clustering-based) is available in literature for homogeneous computing systems [2,12,22,20]; whereas, the related work for HC systems is largely limited to list-based heuristics [9,14,17,18,22,23]. In this technique, tasks are first arranged in a sequence, satisfying precedence constraints, generally based on some priority that is decided by the computation and communication costs of the nodes. The highest priority ready task (a task with satisfied precedence constraints) is then chosen for scheduling followed by the selection of most suitable processor, to accommodate it. Duplication-based techniques, on the other hand, are found to be quite effective for message-passing homogeneous multiprocessor systems [1,3]. This technique, although not explored extensively for HC, holds the potential to generate good quality schedules, which may be at the cost of complexity, at times.

Recently reported SD algorithm [3] blends the simplicity of list-based approach with the performance of duplication-based techniques, and is shown to be quite effective for homogeneous computing systems. It improves the performance of list-based algorithms by incorporating limited yet effective duplication into it. Idle time slots, left in a processor, are exploited for duplicating immediate parents of a child node so as to reduce its waiting time on the processor. Further, some restraints are exercised in duplicating the parent nodes to avoid redundant duplications and superfluous consumption of resources. In this work, we have adapted SD algorithm to heterogeneous environment and then assessed the usefulness of limited duplication approach in dealing with the ‘stresses’ of heterogeneity in a system.

Further, for the sake of presenting the work in a more comprehensive and exhaustive manner, an *A-Cube* (A^3) performance model is proposed, which broadly categorizes the performance metrics, as used by various researchers, into three categories. The model helps in analyzing the behavior of an algorithm, application, and architecture for heterogeneous computing systems. A comparison of state-of-the-art scheduling algorithms is next performed using the suggested

A-Cube model. Scheduling is found to be tougher in HC environment, as the penalty imposed for injudicious moves is more severe in comparison to its homogeneous counterpart.

The work is organized as follows: Section 2 formulates the scheduling problem, and describes the task and machine models used. Related work as available in literature is given in Section 3 along with the salient features of the algorithms used in this work and the proposed algorithm is elaborated. Section 4 states the objectives of *A-Cube* performance model and suggests the relevant performance metrics. Experimental set-up used for performance simulation and task graph generation is discussed in Section 5, with the section following it providing a performance comparison among the state-of-the-art task scheduling algorithms. The subsequent Section 7 discusses and concludes the work with some final remarks.

2. Task and machine models

2.1. Machine model

Heterogeneous processing is an abstract model with wide manifestations. We have categorized these heterogeneity models into two broad categories, as

- Mixed heterogeneity machine model (MHM Model),
- Fixed heterogeneity machine model (FHM Model).

In MHM model, the target machine consists of a mixed suite $P = \{p_k : k = 0, 1, \dots, p-1\}$ of p processors that are best suited to process a particular type of program code. Therefore, execution time of a task t_i on processor p_k will depend on, how well the architecture of p_k matches t_i 's processing requirements. A task scheduled on its best-suited processor will take lesser execution time, as compared to the one scheduled on a less-suited processor. It is to be noted that the best processor for one task may be the worst processor (one that takes maximum execution time/or does not execute at all the particular code type task) for the other. This type of model represents the heterogeneous machine suite as described by Khokhar et al. [10] and covers the type of heterogeneity modeled in [18,19,23], corresponding to a task having different execution times on different processors.

In FHM model, the target machine suite P consists of two subsets of fixed type of homogeneous processors, P_{slow} and P_{fast} , where $P = \{p_k : p_k \in P_{\text{slow}} \text{ or } p_k \in P_{\text{fast}}, \text{ for } k = 0, 1, \dots, p-1\}$. Processors in the two sets differ in their processing powers. However, the ratio of execution time of any two tasks t_l and t_m on any two processors p_k and $p_{k'}$ will remain constant. i.e. $w_{l,k}/w_{l,k'} = w_{m,k}/w_{m,k'}$ (where $w_{i,k}$ refers to the execution cost of a task t_i on processor p_k). A heterogeneity factor

$$\beta = \frac{\text{Number of fast processors in the network}}{\text{Total number of processors in the network}}$$

gives the ratio between numbers of fast processors to the total processors available in the system. This type of heterogeneity model presents a generalized form of the model used by Menasce et al. [15] (with only one fast and rest slow processors), and is intended for HC systems that target the sequential bottleneck part present in the problem by adding few fast (and thus costlier) processors in an otherwise slow homogeneous system.

The term processor power ratio gives the ratio between the processing powers of the fastest and the slowest processor for a task in the network, and is used to take into account, the extent of heterogeneity present in the network for a given task. In the present context for the sake of simplification, it is assumed to be same for all the tasks and is termed as the *computing power ratio* (CPR) for the HC system. Thus, a $CPR = 4$ suggests that there exist in the network a processor that can execute a given task four times faster than the slowest processor. Communication between processors is through *I/O* channels, thereby, allowing concurrent computation and communication. Further, the communication links are assumed contention free and the communication overhead between two tasks scheduled on the same processor is taken as zero.

2.2. Application model

A quadruple Q represents the application model as $Q(T, \mathbf{R}, [c_{ij}], [w_{i,k}])$; $T = \{t_i : i = 0, 1, \dots, v-1\}$ is a set of v tasks; $\mathbf{R}$ represents the precedence relation that defines a partial order on the task set T , such that $t_i \mathbf{R} t_j$ implies, that the task t_i must finish, before t_j can start execution; $[c_{ij}]$ is a $v \times v$ communication cost matrix that gives amount of dataflow between task t_i and t_j for $0 \leq i, j < v$; and $[w_{i,k}]$ is a $v \times p$ matrix of task's computation requirements on different processors for $0 \leq i < v$ and $0 \leq k < p$ (Table 1). A directed acyclic graph (DAG) (Fig. 1) can better represent the application model, with vertices representing the tasks and edges the dataflow paths, and the corresponding weights on them the mean computation cost $\bar{w}_i$ and mean communication costs $\bar{c}_{ij}$, respectively.

$$\bar{w}_i = \frac{\sum_{k=1}^p w_{i,k}}{p}, \quad 0 \leq i < v, \text{ and}$$

$$\bar{c}_{ij} = \frac{c_{ij}}{\text{average rate of data transmission over all links}}.$$

The term CCR refers to the ratio of average communication cost to average computation cost on a given system. A high CCR indicates the communication intensive nature of a problem, whereas, low CCR represents the computation intensive problems. Candidate node/processor refers to the node or processor that is under consideration for scheduling. The objective function of scheduling is to map each task in

Table 1

Computation cost matrix $[w_{i,k}]$ (MHM model)

Task node	Computation time on different processors				$\bar{w}_i$
	p_0	p_1	p_2	p_3	
n_0	1	1	2	1	1.25
n_1	3	2	4	2	2.75
n_2	5	6	3	4	4.5
n_3	2	4	4	2	3.0
n_4	4	8	7	8	6.75
n_5	3	3	1	2	2.25
n_6	5	5	5	5	5.0
n_7	1	2	2	2	1.75

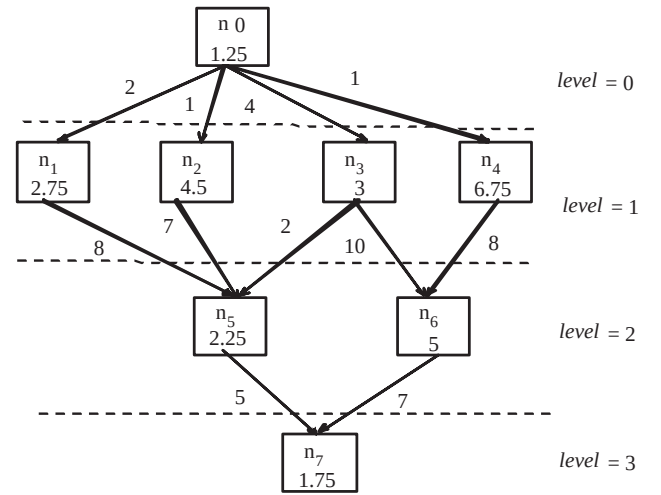


Fig. 1. A simple DAG representing precedence-constrained application graph.

the task graph to an appropriate processor in the processor network and to assign a starting time such that the makespan, i.e. the maximum finish time of a task t_i on a processor p_k ($F_{i,k}$) is minimized.

$$\text{makespan} = \max\{F_{i,k} \mid 0 \leq i < v \text{ and } 0 \leq k < p\}. \quad (1)$$

3. Related work and proposed approach

3.1. Related work

Most of the work available on scheduling of precedence constrained task graphs in HC environment is based on, simple yet effective, list-based heuristics like generalized dynamic level scheduling (GDL) [22], mapping heuristic (MH) [24], heterogeneous earliest finish time (HEFT) [23], critical path on a processor (CPOP) [23], best imaginary level (BIL) [17], and fast critical path (FCP) [18] algorithm. The sound groundwork provided by earlier researches has paved the way for future works in HC; as, most of the scheduling strategies are adopted from the field of homogeneous computing systems. Clustering and

duplication-based heuristics are not much explored for the HC systems. Some recently reported clustering and duplication-based algorithms are Triplet [5], scalable task duplication-based scheduling algorithm (STDS) [19], and leveled duplication-based scheduling algorithms (LDBS) [7]. Salient features of some of these algorithms are discussed here. The high performance HEFT [23] algorithm ($O(pv^2)$), with its roots in the MCP algorithm [24], is an insertion-based static list scheduling heuristic that assigns priorities to the nodes on the basis of *bottom level* (b_i) (sum of mean computation and communication costs along the longest directed path from the concerned node to the exit node). The highest priority ready task is scheduled on a processor that completes its execution at the earliest. Scheduling holes (idle time slots left in a processor due to non-availability of data earlier) are exploited in the process to improve the start time of the candidate node. HEFT algorithm outperforms the GDL, MH, and CPOP algorithms.

CPOP [23] algorithm ($O(pv^2)$) assigns priorities to the nodes on the basis of sum of their *bottom-level* and *top-level* (τ_i) (mean computation and communication costs along the longest directed path from the entry node to the concerned node excluding its computation cost). The algorithm has been developed around the critical path (longest directed path in the DAG in terms of computation and communication costs) reduction scheme. The processor where critical path nodes shall be executed is chosen beforehand, and is the one minimizing their cumulative computation cost. Nodes other than the critical path nodes are scheduled just like the HEFT algorithm. However, we feel that the selection of a processor in advance, for critical path nodes, may not work well, owing to the involvement of precedence and communication cost constraints in calculating the start time of nodes on different processors. It is quite possible that a critical path node may start later on the chosen processor (due to the late data arrival times from its different predecessors) and hence, delaying its completion as well, even if the chosen processor is faster.

STDS [19] algorithm, with its roots in the homogeneous TDS [6] algorithm, is a low complexity ($O(v^2)$) duplication algorithm that picks up an unscheduled task with the lowest *static level* (s_i) (same as *bottom level* but without considering communication costs) and traverses the graph in a bottom-up fashion to generate linear clusters by merging the candidate node with its most favorite predecessor, if it is not scheduled yet; else, the merging is done with a predecessor having minimum execution time on the candidate processor. The initial phase of the algorithm works under the assumption of unbounded availability of processors. In case, number of available processors is less than that required in the clustering phase, fictitious processors are used. The computation cost of a task on the fictitious processor is taken as its mean computation cost. In the second phase, algorithm scales down/up these clusters to the available number of processors by means of compaction/duplication techniques.

However, to reduce the complexity, too much performance is put at stake by the algorithm. For instance, in the du-

plication phase (when the number of available processors are more than the number of clusters generated in the first phase), algorithm picks up the cluster and the task whose predecessors are to be duplicated in a somewhat random fashion. Further, the use of fictitious processors, and the way to get rid of them by merging with *any* processor having adequate free slots raises some doubts about the competitiveness of the algorithm, for the HC environment, owing to the different processing powers of processors.

LDBS [7] algorithms (version 1 and version 2) use a level sorting technique to arrange the nodes in the application DAG into various precedence levels (referred as *level*). The tasks belonging to the same *level* have no data dependencies among them, and hence can be executed concurrently. Tasks are then scheduled *level* by *level* starting from the top (Fig. 1). The nodes in the topmost *level* ($= 0$) are scheduled by using the well-known *min-min* heuristic borrowed from the metatask scheduling techniques [4]. For all other *levels* algorithm does an exhaustive search for the most suitable task-machine pair that gives earliest finish time. Insertion-based duplication technique is employed to further lower the finish times of tasks, and to achieve a performance much better than the HEFT algorithm. However, the complexity of the two algorithms is quite high $O(p^2ev^3)$ and $O(p^3ev^3)$ where ' e ' corresponds to the number of edges in the DAG. Further, search for the most suitable task-machine pair in LDBS₁ (version 1) algorithm may overlook the priority of a task. A low priority task may get scheduled earlier if it finds a better match in the network. LDBS₂ algorithm (version 2) does rectify it by scheduling the tasks in a given *level* in the order of their *bottom-level* priority but, due to the leveled approach as adopted by both of the LDBS algorithms, this priority gets localized to a particular precedence level, which may not reflect the true priority for scheduling a task.

3.2. Heterogeneous limited duplication algorithm

Keeping in view the pros and cons of available algorithms, insertion-based selective duplication approach of SD algorithm [3] seems to be quite attractive for HC as it exploits scheduling holes, employs selective duplications and can work with any of the priority scheme due to its generic nature. Furthermore, it is conjectured that the economical utilization of scheduling holes will be more relevant and significant in HC environment, as the high-speed and best-suited resources in a heterogeneous network are limited. Any extravagant consumption of these resources can deprive a task from using them and in the worst case, a task may get settled on a least-suited processor thereby affecting the overall performance of an algorithm. A low complexity heterogeneous limited duplication (HLD) scheduling algorithm, an adaptation of the SD algorithm [3] for the HC environment, is presented here with performance comparable or better than the LDBS algorithm. It schedules the tasks strictly in the order of their global priority, and restricts duplications to the most

```

HLD Algorithm
Begin
Step 1: Construct a priority based task sequence  $\xi$ ;
do
Step 2:  $t_i \leftarrow$  first unscheduled task in  $\xi$ ;
Step 3: for (all  $p_k$  in  $P$ )
        Call  $\text{Get\_EFT}(t_i, p_k)$  and record the  $F_{i,k}$ ;
Step 4: Schedule  $t_i$  on processor  $p_k$  with minimum  $F_{i,k}$ ;
        while (there exists an unscheduled task in  $\xi$ )
End
-----
Get_EFT ( $t_i, p_k$ )
Begin
  Repeat
  {
Step s1:  $S_{i,k} \leftarrow$  Find start time of  $t_i$  on  $p_k$ ;
Step s2:  $F_{i,k} \leftarrow S_{i,k} + w_{i,k}$ ;
Step s3:  $M_i \leftarrow$  Find MIIP of  $t_i$  for  $p_k$ ;
Step s4: if ( $M_i$  does not exist or is already scheduled on  $p_k$ )
        Return  $F_{i,k}$ ;
      else if (suitable slot exists for  $M_i$  on  $p_k$ )
        {  $F_{j,k} \leftarrow$  Find the finish time of  $M_i (= t_j \text{ say})$  on  $p_k$ ;
          if ( $F_{j,k}$  is less than  $S_{i,k}$ )
            Duplicate  $M_i$  on  $p_k$ ; // duplication successful, repeat the loop for next MIIP
          else Return  $F_{i,k}$ ;
        }
      else Return  $F_{i,k}$ ;
  }
End

```

Fig. 2. Pseudo code for HLD algorithm.

crucial immediate parents so as to avoid redundant replications. The pseudo code of the algorithm is shown in Fig. 2. The algorithm completes in two main phases as described in the following sections.

3.2.1. Priority-based task sequence generation

Priorities are assigned to the nodes on the basis of their *bottom-level* that is calculated recursively using mean cost parameters as

$$b_i = \bar{w}_i + \max\{b_j + \bar{c}_{ij}\}, \quad \forall t_j \in \text{succ}(i). \quad (2)$$

The term $\text{succ}(i)$ refers to the set of immediate child nodes of t_i in the DAG. Arranging the nodes in non-increasing order of their *bottom-level* automatically sorts them in the topological order and generates a task sequence ξ satisfying inter-task precedence. Our decision behind choosing the *bottom-level* priority is partly based on its good track record (for homogeneous as well as heterogeneous scheduling algorithms) but, more than that it will help us in quantifying the performance of HLD algorithm over the HEFT algorithm, which uses the same priority scheme.

3.2.2. Task assignment and duplication phase

In this phase, tasks are assigned to the processors and duplication is employed to reduce their start and hence finish

times. First unscheduled task in the task sequence is selected and scheduled on a processor that can complete its execution at the earliest using duplication. It is to be noted that this processor may or may not be its best-suited processor. This type of heuristic automatically takes into consideration the computing power of the candidate processor. Free time slots, if any, in the processor's schedule list are exploited in this process to accommodate the candidate node and duplicated nodes to facilitate their early start as explained below.

To start execution on a processor (p_k , say) the task t_i has to wait for the data arrivals from all of its immediate parents ($\text{pred}(i)$), so as to honor the precedence constraints. The parent whose data arrives last of all is termed as the most important immediate parent (MIIP) of t_i , as it blocks the earlier start of the task t_i . Data arrival time for t_i on p_k is given by

$$\text{DAT}(t_i, p_k) = \max_{t_y \in \text{pred}(i)} \{\min\{F_{y,k}, F_{y,k'} + c_{yi}\}\} \quad (3)$$

$p_{k'} =$ processor that hosts originally scheduled t_y .

Due to the non-availability of data earlier, owing to precedence-constraints in task graph or communication delays in the processor network, a processor may remain idle leading to the formation of scheduling holes. The algorithm

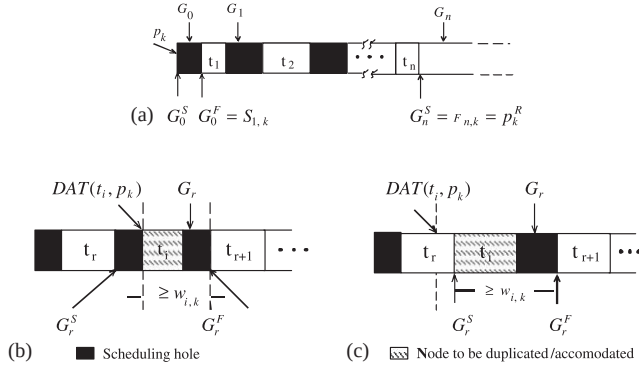


Fig. 3. (a) Free slot description and suitability of a free slot (G_r) to accommodate a node (t_i); (b) data arrival time of the node is greater than the start time of the slot; (c) data arrival time of the node is less than or equal to the start time of the slot.

exploits suitable scheduling holes as available on p_k for replicating the MIIP, so as to improve its data arrival time. Suitability of a free slot to accommodate or duplicate a task is decided as per Definition 1.

Definition 1. Given a set of n tasks $\{t_1, t_2, \dots, t_n\}$ scheduled on a processor p_k , a free time slot (gap) G_r (between t_{r+1} and t_r), with G_r^S and G_r^F as its start and finish time, respectively, $0 \leq r \leq n$, is suitable for task t_i if,

$$G_r^F - \max\{DAT(t_i, p_k), G_r^S\} \geq w_{i,k}, \text{ where} \\ G_0^S = 0, G_0^F = S_{1,k}, G_n^S = F_{n,k}, \text{ and } G_n^F = \infty.$$

The start time ($S_{i,k}$) of t_i on p_k is thus limited by the data arrival time of its MIIP (say M_i) and the availability of a suitable scheduling hole. In case of non-availability of a suitable slot, the task t_i shall start after the last task scheduled on p_k completes its execution, i.e. at the ready time (p_k^R) of processor p_k (Fig. 3(a)). The start time of t_i on p_k is given by Eq. (4)

$$S_{i,k} = \max\{DAT(M_i, p_k), \min\{p_k^R, G_r^S\}\}, \quad (4)$$

where G_r corresponds to the first suitable slot on p_k .

Unlike the SD algorithm, here we are interested in improving the finish time of t_i on p_k ($F_{i,k}$) to take into account the impact of heterogeneous processing powers of different processors. The finish time of t_i is first calculated without duplications, and then possibility of duplicating its MIIP is explored (Step s4 in Fig. 2). To avoid redundant duplications, an MIIP is duplicated on the candidate processor only if it is able to supply the requisite data to the candidate task earlier than that without duplication, so as to save the precious resources for later utilizations. Finish time is recalculated after duplication for recording possible improvements and the process is repeated till there is no more MIIP, or no suitable slot is available, or MIIP is already scheduled on p_k . The process is repeated for all the processors and t_i is scheduled on a processor that gives earliest finish time.

$$F_{i,k} = S_{i,k} + w_{i,k}. \quad (5)$$

3.2.3. Running trace and complexity of the algorithm

Running trace and schedule generated by the algorithm, for the example DAG shown in Fig. 1, is given in Table 2 and Fig. 4(a) in comparison to the HEFT algorithm (Table 3 and Fig. 4(b)), which uses similar priority and insertion-based scheduling approach, but no duplication. Replication of nodes helps HLD algorithm to start early execution of nodes n_1, n_3, n_5 , and n_7 , which helps in generating a makespan equal to 16 in comparison to 20 of the HEFT algorithm. The performance of HLD algorithm in comparison to HEFT algorithm directly quantifies the benefits gained by adding limited duplication in an algorithm.

The overall worst-case complexity of HLD algorithm comes out to be $O(pv^2d_{\max} + e)$ where $d_{\max}$ is the maximum in/out degree of a task in the DAG. The complexity of task sequence generation phase is $O(v + e + v \log v)$, and that of task assignment and duplication phase is $O(pv^2d_{\max})$. In comparison to LDBS algorithms ($O(p^2ev^3)$ and $O(p^3ev^3)$), complexity of HLD algorithm is lower by an order at least. It can be seen from the strategies adopted by the two algorithms.

LDBS₁ algorithm does an exhaustive search among all the tasks within a given *level* to look for the most suitable task-machine pair, while scheduling each and every task. It amounts to an increase in complexity by an order of $O(v)$. In the LDBS₂ (version 2) algorithm, tasks within a *level* are sorted with *bottom level* priority and then scheduled one by one, thereby improving upon the complexity front to some extent. However, both of these algorithms consider all the available processors along with the candidate processor for duplicating a *critical node* (same as MIIP) and look for its impact on the start time of the candidate task on the candidate processor. It results in further incrementing the complexity by $O(p)$ at least. However, we conjecture that search for processors other than the candidate processor, for possible duplication of MIIPs, may not deliver desired results due to the communication costs involved. Further, the search for the most suitable task-machine pair may overlook the priority of a task as discussed earlier. HLD algorithm, in contrast, schedules tasks strictly in the order of their global priority and considers only the candidate processor for duplicating the MIIPs, thereby improving upon the complexity front without sacrificing much on performance.

4. Performance model

Performance measures demand a prudent thought and should be designed keeping in view the basic objectives behind HC. For the sake of fair comparison, besides using identical task and machine models, it is desirable to analyze algorithms from various aspects. Most of the algorithms have either been tested for MHM or FHM model that too with limited performance parameters. An A-Cube performance model is proposed with the objective of studying the behavior of an Algorithm, Application, and Architecture (A³),

Table 2
Running trace of the HLD algorithm (MHM model, $p = 4$)

Node	bottom-level b_i	Task sequence ζ	Finish time of a task on different processors				Node/s duplicated
			p_0	p_1	p_2	p_3	
n_0	32.0	n_0	1 ^a	1	2	1	nil
n_1	19.75	n_4	5 ^a	9	9	9	nil
n_2	20.5	n_3	7	5	6	3 ^a	n_0
n_3	26.75	n_2	10	7	5 ^a	7	nil
n_4	28.75	n_1	8	3 ^a	9	5	n_0
n_5	9.0	n_6	12 ^a	18	18	16	n_3
n_6	13.75	n_5	15	12	10 ^a	11	n_1
n_7	1.75	n_7	16 ^a	20	20	20	nil

^aEarliest finish time selected.

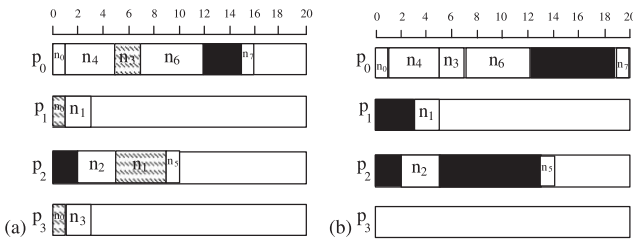


Fig. 4. Schedule generated by (a) HLD algorithm (makespan = 16) and (b) HEFT algorithm (makespan = 20).

in the presence of heterogeneity. More specifically, through this model we try to analyze performance with an objective to answer the following:

- How does a scheduling algorithm behave under different heterogeneity models, for various task graph and heterogeneity parameters?
- For a given algorithm and heterogeneity model, how does an application benefit from the extent of heterogeneity (CPR)?
- How does an algorithm perform on two cost equivalent homogeneous and heterogeneous architectures?

Performance metrics used for the purpose are normalized schedule length (NSL), makespan ratio (MSR) and architectural speedup (A_{spd}), respectively.

$$(i) \quad NSL = \frac{\text{Makespan}}{\max\{s_i\}} \quad \text{for} \quad t_i \in Q_{\min} \quad \text{and} \quad 0 \leq i < v. \quad (6)$$

NSL gives makespan generated by an algorithm normalized with respect to its absolute lower bound. For calculating this absolute lower bound, we assume that every node in the given DAG is processed by its best-suited processor; such a hypothetical DAG is referred as $Q_{\min}$, as it corresponds to the minimum computation costs for all the tasks. Next, we look in for the longest directed path (considering computation costs) in $Q_{\min}$ by calculating *static-levels* for all the tasks. Since the nodes along a directed path are to be executed sequentially, the denominator in Eq. (6), gives the

absolute lower bound on the makespan for the given DAG. A lower NSL value is always desirable.

$$(ii) \quad MSR = \frac{\text{Makespan}_{HT}}{\text{Makespan}_{HM}}. \quad (7)$$

Makespan ratio (MSR) gives the ratio of makespans generated by an algorithm for a heterogeneous system (with $CPR \neq 1$) and a homogeneous system ($CPR = 1$). If the objective of adding heterogeneity in a system is to improve the makespan of an application, then an algorithm generating MSR less than one is desirable. A value greater than or equal to one indicates that the algorithm is unable to improve upon its homogeneous counterpart.

$$(iii) \quad A_{spd} = (\text{MSR}_{\text{costeq}})^{-1}. \quad (8)$$

The architectural speedup A_{spd} parameter [15] compares the performance of algorithm on cost equivalent homogeneous and heterogeneous computing architectures. It is the ratio of makespans of the algorithm on a homogeneous system and on its cost equivalent heterogeneous system. For cost equivalence, Menasce [15] used p number of slow processors in the homogeneous system, if the HC system has one fast plus $(p - \alpha)$ identical slow processors; α being the ratio of processing powers of the fast and the slow processor (heterogeneity model adopted was FHM, with only one fast processor). This relation was based on the constant returns cost function that says cost of a machine is proportional to its capacity [16]. We have modified the expression for the generalized FHM model as used by us. The relation between number of processors in a homogeneous system (p_{HM}), and its cost equivalent heterogeneous system (p_{HT}) in terms of heterogeneity factor β is given as

$$p_{HT} = \left\lfloor \frac{p_{HM}}{1 + \beta(CPR - 1)} \right\rfloor. \quad (9)$$

An algorithm with A_{spd} more/less than one reflects the superiority/inferiority of heterogeneous architecture over homogeneous architecture.

Table 3
Running trace of the HEFT algorithm (MHM model, $p = 4$)

Node	bottom-level b_i	Task sequence ζ	Finish time of task on different processors				Node/s duplicated
			p_0	p_1	p_2	p_3	
n_0	32.0	n_0	1 ^a	1	2	1	-not applicable-
n_1	19.75	n_4	5 ^a	10	12	13	
n_2	20.5	n_3	7 ^a	9	9	7	
n_3	26.75	n_2	12	8	5 ^a	6	
n_4	28.75	n_1	10	5 ^a	9	5	
n_5	9.0	n_6	12 ^a	22	22	22	
n_6	13.75	n_5	16	15	14 ^a	15	
n_7	1.75	n_7	20 ^a	21	21	21	

^aEarliest finish time selected.

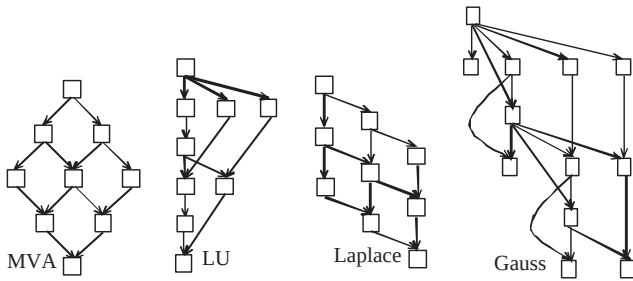


Fig. 5. Elementary task graph structures for some regular parallel numerical applications.

5. Task graph generation

For an unbiased evaluation of algorithms, performance is simulated for random and regular graphs, which are generated by modifying the corresponding benchmark task graph suites [12], used for homogeneous systems, so as to go with our MHM and FHM models. The random task graph suite consists of 5000 graphs; ten different sizes from 50 to 500 with an increment of 50; five different CCRs 0.1, 0.5, 1.0, 2.0, 10.0; five different average parallelism (largest number of non-precedence related nodes in the graph) as $n\sqrt{v}$ ($1 \leq n \leq 5$), and 10 CPR values from 1 to 10 for each of the two models. The regular task graph suite corresponds to various parallel numerical applications; Cholesky decomposition (Cholesky), Gaussian elimination (Gauss), LU decomposition (LU), Mean Value Analysis (MVA), and Laplace equation solver (Laplace). The elementary structures of some of these applications are shown in Fig. 5. As all these applications operate on matrices, so the task graph size is roughly $O(N^2)$; N varies from 19 to 64 for the former three applications, and from 19 to 46 for the latter two. Here again 2500 regular graphs (5 applications $\times$ 10 sizes $\times$ 5 CCRs $\times$ 10 CPRs) are generated for each heterogeneity model. CCR is varied in the same proportion as in random graph suite so as to have wide ranging task graph granularities.

In total, 10,000 random and regular graphs are generated. A CPR value equal to one represents a homogeneous system

and is included for the sake of comparison. The original computation and communication costs in the random and regular benchmark graph suites [12] are taken as the mean computation $\overline{w_i}$ and communication costs $\overline{c_{ij}}$, respectively, in the beginning. The estimated execution time of a task t_i on a processor p_k , i.e. $w_{i,k}$ is calculated as in Eqs. (10) and (11) for FHM and MHM models, respectively.

$$w_{i,k} = \begin{cases} \frac{2 \times \text{CPR}}{1 + \text{CPR}} \times \overline{w_i} & \text{if } p_k \in P_{\text{fast}}, \\ \frac{2}{1 + \text{CPR}} \times \overline{w_i} & \text{if } p_k \in P_{\text{slow}}, \end{cases} \quad (10)$$

$$\frac{2}{1 + \text{CPR}} \times \overline{w_i} \leq w_{i,k} \leq \frac{2 \times \text{CPR}}{1 + \text{CPR}} \times \overline{w_i}. \quad (11)$$

The mean communication and computation costs were modified, wherever necessary, to maintain the corresponding CCR parameters. The number of processors was taken as an input parameter and was kept same for all the algorithms during the various simulation runs, and heterogeneity factor β was taken as 0.1 for the FHM model.

6. Experimental results and performance comparison

Performance of HLD algorithm is presented in comparison to four recently proposed heterogeneous scheduling algorithms. These are list-based HEFT and CPOP algorithms [23], and duplication-based STDS¹ [19] and LDBS (version 2) [7] algorithms. An explicit comparison to some other well-known heterogeneous algorithms like GDL [22], MH [24], and BIL [17], is not undertaken as the chosen algorithms have already been tested against them, and are found to generate comparable or better schedules. The results, wherever presented, are in the order STDS, CPOP, HEFT, LDBS, and HLD.

¹ We thank the authors for providing implementation of their STDS code.

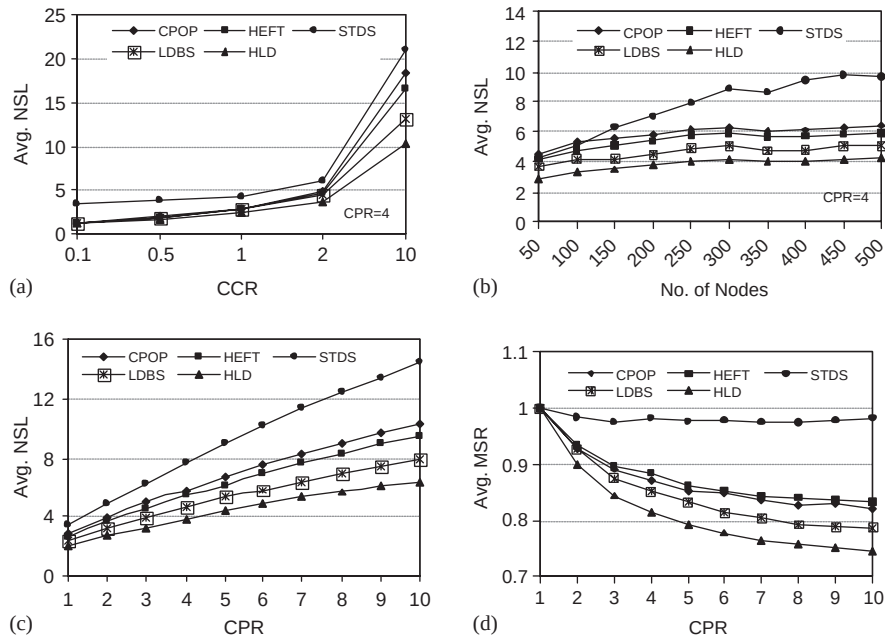


Fig. 6. Performance comparison for the random graph suite using MHM model.

6.1. Performance for random graphs

6.1.1. MHM model

For a given CPR value, the effect of task graph parameters on the algorithms is presented in Fig. 6 (Figs. 6(a) and (b)). Average NSL tends to deteriorate with the increasing CCR and number of nodes, because of the trade-off between available parallelisms and inter process communication. Duplication-based algorithms (HLD and LDBS) try to retain this parallelism in spite of high communication costs by replicating the parent nodes and thereby, performing better at higher CCRs.

Results in Fig. 6(c) (averaged over the complete MHM random graph suite) show the effect of heterogeneity on the performance of algorithms. On an average performance is found to deteriorate with increasing heterogeneity. This observation reflects that any misjudgment or lack of farsightedness on the part of heuristic may prove more costly in an HC environment, as the task may get scheduled on a less suitable processor (unlike homogeneous system where all the processors are identical). Duplication based techniques, however, succeeds to some extent in overcoming these injudicious moves, whereas in the list-based schemes there is no way to rectify or partly compensate for them at any later stage. In comparison to LDBS, HEFT, CPOP, and STDS algorithms, average NSL generated by HLD algorithm is better by as much as 18%, 30%, 36% and 52%, respectively.

The effect of heterogeneity on the application is presented in Fig. 6(d) in terms of average MSR. Average ($2 \leq \text{CPR} \leq 10$) makespan improvement for the HC systems in comparison to homogeneous system is about 2%, 14%,

14%, 17% and 21%, respectively. HLD algorithm is better able to adjust to the HC environment and delivers more benefits of heterogeneity to the application.

6.1.2. FHM model

Performance for the fixed heterogeneity machine model is presented in Fig. 7. Scheduling in FHM model is found to be tougher and the penalty imposed for injudicious scheduling is more severe as is clear from the corresponding results shown in Figs. 7(a)–(c). FHM model tends to restrict the parallelism especially at higher CPR values, as all tasks start competing for the few fast processors. Average NSL degradation suffered by various algorithms with respect to MHM model is about 79%, 54%, 20%, 1%, and 8%.

LDBS algorithm that makes an exhaustive task-machine pair search, performed quite consistently for both the models. HLD algorithm, which restricts this search for the sake of complexity, competes quite closely with it. In comparison to LDBS, HEFT, CPOP and STDS algorithms, average NSL generated by HLD algorithm is better by 11%, 37%, 54% and 71%, respectively.

Average MSR results (Fig. 7(d)) are found to be quite sensitive to the type of heuristic for the chosen heterogeneity model. MSR improvements are about –49%, –14%, +10%, +25% and +23%. STDS and CPOP algorithms generated schedules longer than their homogeneous counterpart, for the tested random graph suite, while HLD algorithm performed next to the LDBS algorithm though with much lower time complexity.

Average architectural speedup (Fig. 7(e)) is again better for HLD and LDBS algorithms. These results indicate how a given scheduling heuristic can add value to a given archi-

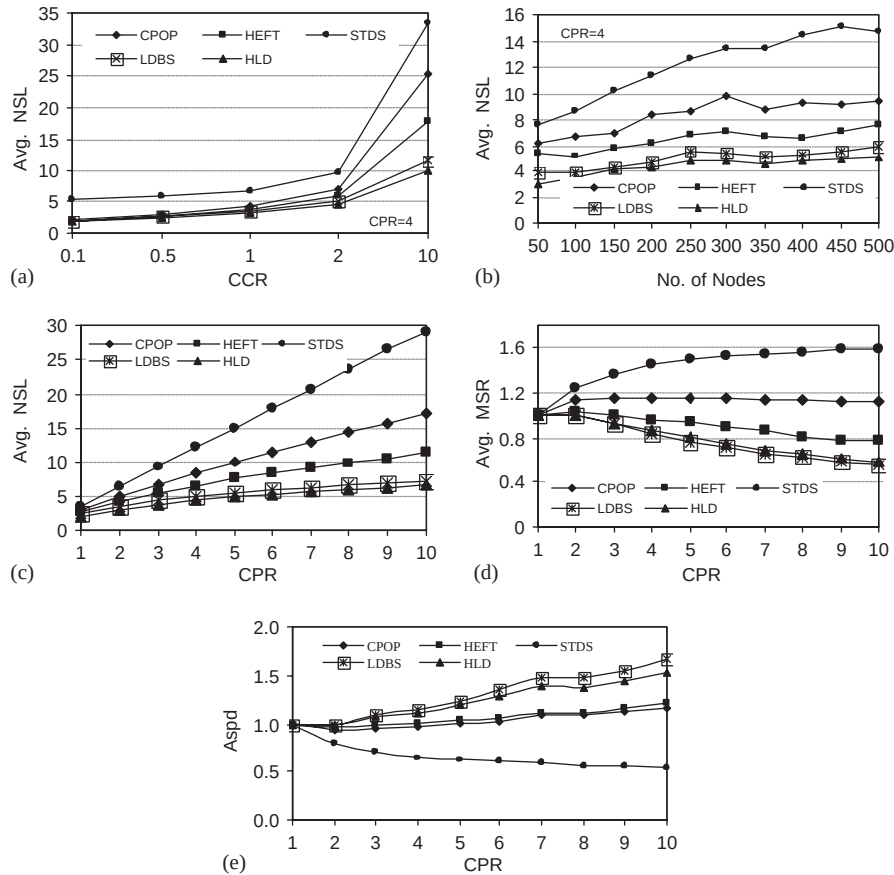


Fig. 7. Performance comparison for the random graph suite using FHM model.

texture. Duplication-based HLD and LDBS algorithms outperformed the list-based algorithms considerably.

6.2. Performance for regular graphs

Simulation results for all the regular graph types showed degradation in algorithm's performance with increasing size, CCR and CPR. As the results cannot be detailed for each and every application, so, the most relevant results are summed up in Table 4 that features average NSL (over $1 \leq \text{CPR} \leq 10$) degradation suffered, by the tested algorithms, with respect to the HLD algorithm. Performance of HLD algorithm is quite consistent and better than others in most of the cases.

However, it is interesting to note that in the FHM model, for the MVA application, performance of list-based (HEFT and CPOP) algorithms is better than the duplication-based (HLD and LDBS) algorithms. This is attributed to the peculiar nature of MVA task graph (Fig. 5) that has highly intricate task dependencies with all paths as critical paths. Further, the task parallelism is expanding in the first half of the DAG and contracting in the later half. HLD algorithm, which exploits limited duplication, is not able to cope with this abrupt change in parallelism and shows somewhat poor performance.

Table 4
(%) Average NSL degradation with respect to HLD algorithm

Graph type	Algorithm			
	LDBS (%)	HEFT (%)	CPOP (%)	STDS (%)
<i>MHM model</i>				
Cholesky	2	23	27	47
Gauss	2	33	41	318
Laplace	4	21	19	102
LU	1	21	24	46
MVA	4	36	25	150
Random	21	44	55	109
<i>FHM model</i>				
Cholesky	1	20	26	128
Gauss	3	22	28	819
Laplace	0	21	5	147
LU	0	23	27	107
MVA	4	-13	-3	351
Random	12	59	120	244

Performance comparison of algorithms in terms of average MSR is shown in Fig. 8. It may be recalled that an MSR greater than one indicates longer completion time of an application for the heterogeneous network in comparison to homogeneous network. Depending on the heterogeneity model and extent of heterogeneity, makespan of an applica-

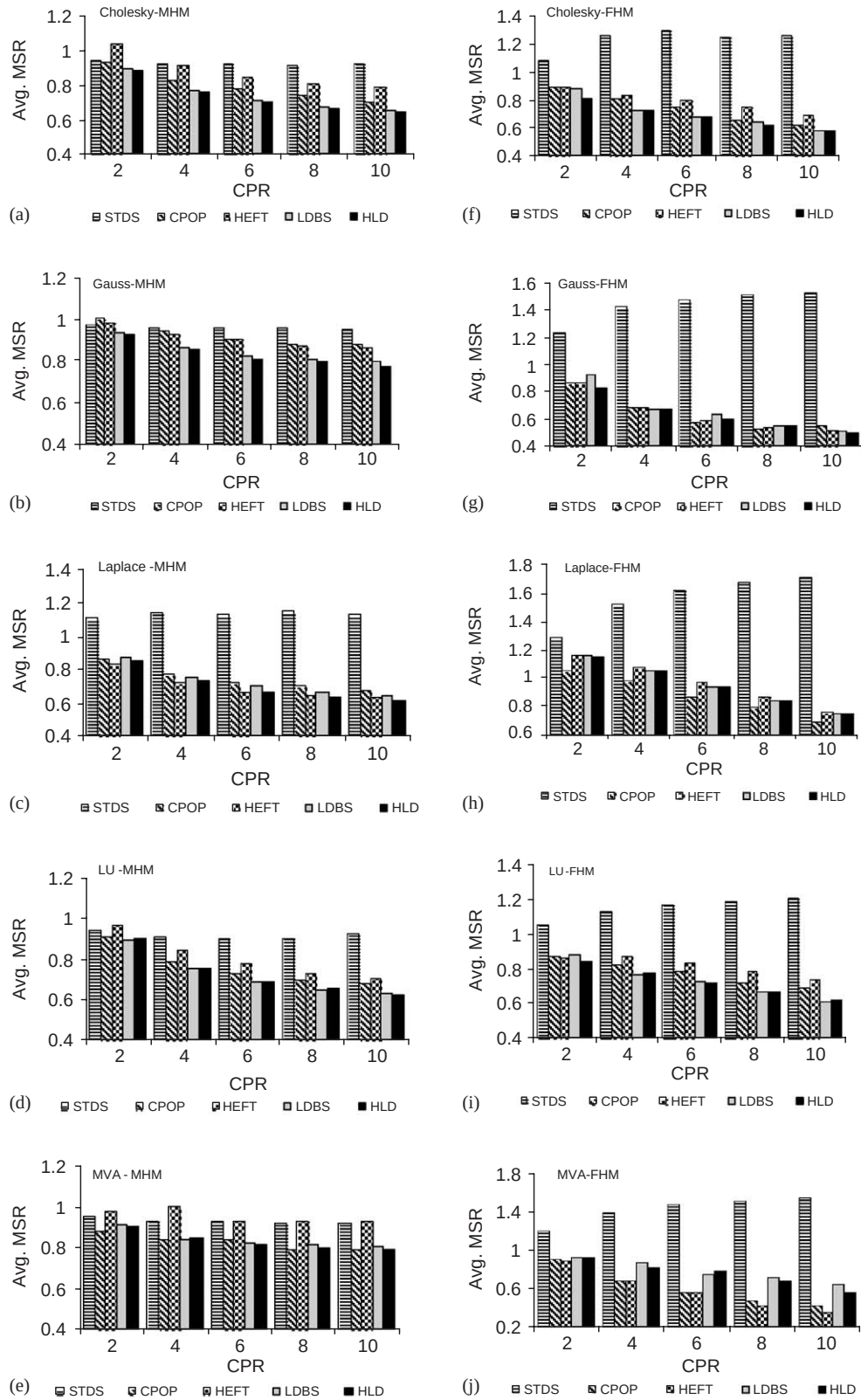


Fig. 8. Effect of heterogeneity on the makespan of regular applications for the MHM and FHM models.

tion tends to improve in general, though not in a consistent manner.

Duplication-based algorithms deliver better performance, with HLD algorithm at the forefront; however, STDS algorithm that first generates schedules assuming unbounded availability of processors and then scales it down to the available number of processors could not take much advantage of heterogeneity and duplication, especially under the FHM model and suffered maximum degradations as shown in Fig. 8.

7. Discussions and conclusions

Heterogeneous processing has wide manifestations. We have broadly categorized heterogeneous architectures under mixed heterogeneity machine model and fixed heterogeneity machine model. MHM model captures the system heterogeneity designed for combating diverse parallelism issue, whereas, the latter deals with sequential bottleneck part of the problem. Further, we have extended the selective duplication approach [3], which improves the performance of list-based algorithms with the addition of limited and effective duplication, for HC environment. The straightforward duplication approach, confined to only immediate predecessors in the graph and the candidate processor in the network, gives the suggested HLD algorithm an edge over the more complex duplication strategy employed in some recently reported algorithms. Further, an *A-Cube* performance model is suggested to study the effect of heterogeneity, logically and comprehensively, on the algorithm's performance, application's makespan, and on architectural speedup. A comparison of state-of-the-art heterogeneous scheduling algorithms is next carried out with widely varying heterogeneity and task graph parameters. However, since contention and network loading effects have not been taken into consideration, these results may be taken as indicative, and not evidential, of the comparative performance of state-of-the-art algorithms in an HC environment.

Performance of all the scheduling algorithms tends to degrade in the presence of heterogeneity. This degradation becomes more pronounced with increase in heterogeneity and at higher CCRs. The makespan of an application benefits in general from heterogeneity, although, the kind of heuristic selected, the task graph structure, and the type and extent of heterogeneity play a crucial role in it. Cost effectiveness of an underlying HC architecture is influenced by the choice of scheduling heuristic.

Judicious utilization of limited resources is highly desirable for the HC scheduling algorithms. Depending upon the type and amount of heterogeneity, performance degradation resulting out of inappropriate task/processor selection tends to be more severe for HC in comparison to its homogeneous counterpart. Any mistake on the part of heuristic may prove costly as the task may have to settle for a less suitable processor. In list-based schemes, there is no way to rectify or

partly compensate for these injudicious/erroneous moves at any later stage. The proposed duplication technique, however, succeeds substantially in overcoming these 'stresses' and 'strains' of heterogeneity by replicating the more crucial nodes for a task, and thereby improving its start/finish time on processors other than the few selected ones. As a result, the heuristic performs appreciably well in HC environment for both the heterogeneity models with a modest increase in complexity. However, in comparison to other duplication-based algorithms, complexity is kept in check by limited yet effective duplication, which retains the best features of duplication without compromising much on the performance and complexity fronts.

Acknowledgments

Authors extend their heartiest gratitude to the anonymous reviewers for their useful and critical comments and to the All India Council for Technical Education, for their Quality Improvement Program under MHRD, Government of India.

References

- [1] I. Ahmad, Y.-K. Kwok, On exploiting task duplication in parallel program scheduling, *IEEE Trans. Parallel Distrib. Systems* 9 (9) (September 1998) 872–892.
- [2] S. Bansal, P. Kumar, K. Singh, Duplication based scheduling algorithm for interconnection constrained distributed memory machines, *Proceedings of the Ninth International Conference on High Performance Computing (HiPC-2002)*, India, Lecture Notes in Computer Science, vol. 2552, December 2002, pp. 52–62.
- [3] S. Bansal, P. Kumar, K. Singh, An improved duplication strategy for scheduling precedence constrained graphs in multiprocessor systems, *IEEE Trans. Parallel Distrib. Systems* 14 (6) (June 2003) 533–544.
- [4] T.D. Braun, H.J. Siegel, et al., A comparison study of static mapping heuristics for a class of meta-tasks on heterogeneous computing systems, in: *Proceedings of the International Heterogeneous Computing Workshop (HCW'99)*, April 1999, pp. 15–29.
- [5] B. Cirou, E. Jeannot, Triplet: a clustering scheduling algorithm for heterogeneous systems, in: *Proceedings of the International Conference on Parallel Processing Workshop (ICPPW)*, September 2001, pp. 231–236.
- [6] S. Darbha, D.P. Agrawal, Optimal scheduling algorithm for distributed memory machines, *IEEE Trans. Parallel Distrib. Systems* 9 (1) (January 1998) 87–95.
- [7] A. Dogan, F. Ozguner, LDBS: a duplication based scheduling algorithm for heterogeneous computing systems, in: *Proceedings of the International Conference on Parallel Processing (ICPP-2002)*, Canada, August 2002, pp. 352–359.
- [8] R.F. Freund, H.J. Siegel, Heterogeneous processing, *IEEE Comput.* 26 (6) (June 1993) 13–17.
- [9] M.A. Iverson, F. Ozguner, G.J. Follen, Parallelizing existing applications in a distributed heterogeneous environment, in: *Proceedings of the Fourth Heterogeneous Computing Workshop*, Santa Barbara, CA, April 1995, pp. 93–100.
- [10] A.A. Khokhar, V.K. Prasanna, M.E. Shaaban, C.-L. Wang, Heterogeneous computing: challenges and opportunities, *IEEE Comput.* 26 (6) (June 1993) 18–27.

- [11] M. Kumar, Measuring parallelism in computation-intensive scientific/engineering applications, *IEEE Trans. Comput.* 37 (9) (September 1988) 1088–1098.
- [12] Y.-K. Kwok, I. Ahmad, Benchmarking and comparison of the task graph scheduling algorithms, *J. Parallel Distrib. Comput.* 59 (3) (December 1999) 381–422.
- [13] R.E. Lord, J.S. Kowalik, S.P. Kumar, Solving linear algebraic equations on an MIMD computer, *J. ACM* 30 (1) (January 1983) 103–117.
- [14] D.A. Menasce, S.C.S. Porto, P. Rio, Scheduling on heterogeneous message passing parallel architectures, *J. Comput. Software Eng.* 1 (3) (1993).
- [15] D.A. Menascé, D. Saha, S.C.S. Porto, V.A.F. Almeida, S.K. Tripathi, Static and dynamic processor scheduling disciplines in heterogeneous parallel architectures, *J. Parallel Distrib. Comput.* 28 (1) (1995) 1–18.
- [16] H. Mendelson, Economies of scale in computing: Grosh's law revisited, *Commun. ACM* 30 (12) (December 1987) 1066–1072.
- [17] H. Oh, S. Ha, A static heuristic for heterogeneous processors, *Euro-Par'96*, vol. II, Lyon, France, August 1996, pp. 573–577.
- [18] A. Radulescu, A.J.C. van Gemund, Fast and effective task scheduling in heterogeneous systems, in: *Proceedings of the Ninth International Heterogeneous Computing Workshop*, 2000.
- [19] S. Ranaweera, D.P. Agrawal, A scalable task duplication based scheduling algorithm for heterogeneous systems, in: *Proceedings of the International Conference on Parallel Processing*, Toronto, Canada, August 2000, pp. 383–390.
- [20] H. El-Rewini, T.G. Lewis, H.H. Ali, *Task Scheduling in Parallel and Distributed Systems*, Prentice-Hall, Englewood Cliffs, NJ, 1994.
- [21] H.J. Siegel, J.K. Antonio, R.C. Metzger, M. Tan, Y.A. Li, Heterogeneous computing, in: A.Y. Zomaya (Ed.), *Parallel and Distributed Computing Handbook*, McGraw-Hill, New York, 1996, pp. 725–761.
- [22] G.C. Sih, E.A. Lee, A compile-time scheduling heuristic for interconnection-constrained heterogeneous processor architectures, *IEEE Trans. Parallel Distrib. Systems* 4 (2) (February 1993) 175–187.
- [23] H. Topcuoglu, M.-Y. Wu, Performance-effective and low complexity task scheduling for heterogeneous computing, *IEEE Trans. Parallel Distrib. Systems* 13 (3) (March 2002) 260–274.
- [24] M.-Y. Wu, D.D. Gajski, Hypertool: a programming aid for message-passing systems, *IEEE Trans. Parallel Distrib. Systems* 1 (3) (July 1990) 330–343.



Savina Bansal received her BSc degree from Punjab university in 1984, BE (Hons) degree in electronics and electrical communication from Punjab Engineering College Chandigarh in 1988, ME (Hons) degree in computer science from TIET Patiala in 1994 and Ph.D. degree from IIT Roorkee, India in 2004. Presently she is with the faculty of electronics engineering department of GZS College of Engineering & Technology, Bathinda (Punjab), India, since 1993. Her current areas of interest include fault tolerant interconnection networks, parallel and distributed computing and multiprocessor scheduling. She is a senior member of Computer Society of India.



Padam Kumar (M'02) born in 1949 in Uttar Pradesh, India, obtained his B.Tech and M.Tech degrees in Electrical Engineering from Indian Institute of Technology Delhi, India, in 1970 and 1972, respectively. Subsequently he joined University of Roorkee (now IIT Roorkee) as Lecturer and received his Ph.D. degree in Computer Science from there. Currently he is a Professor of Computer Science in the Department of Electronics and Computer Engineering at IIT Roorkee. He has supervised 5 Ph.D.

scholars and more than 60 MS students. His areas of interest include parallel processing, multiprocessor scheduling, real-time systems and fault-tolerant computing. He is a member of IEEE.



Kuldip Singh received his bachelor and master's degrees in electronics and communication engineering, and Ph.D. degree in computer science, from the University of Roorkee. He joined the University of Roorkee (now, IIT Roorkee) in 1973 as a lecturer. Presently, he is a professor in computer engineering at IIT Roorkee. His areas of interests are parallel processing and computer communication networks. Dr. Singh has published more than 40 technical papers in journals and conference proceedings and organized several national and international conferences/workshops. He is also the founder chairman of the editorial board of Indian Journal of Continuing Engineering Education.